

NESEmu - Nintendo Entertainment System Emulator



Oliver Long
Computer Science with Foundation Year in Computing

ABSTRACT

This project presents the design and implementation of a Nintendo Entertainment System (NES) emulator developed as a teaching tool for understanding low level computing systems. The emulator replicates key hardware components, including the 6502-based CPU, Picture Processing Unit (PPU), and system bus, allowing NES software to be executed in a controlled and observable environment.

The design is based around established documentation of NES hardware behaviour, particularly the NESDev Wiki (NESDev Wiki, n.d.), which provides detailed descriptions of CPU instruction sets, memory mapping, and graphical rendering. A modular architecture was used, enabling separation of concerns between components and reflecting object oriented design outlined by Gamma (1995). Development followed an iterative approach aligned with Agile methodologies (Beck et al, 2001), allowing incremental implementation and testing.

A key contribution of the project is the integration of real time debugging tools, including memory viewers, register displays, and graphical visualisations, which enhance understanding of internal system behaviour. The emulator was validated using NES test ROMs and demonstrated accurate CPU execution and graphical output. Overall, the project combines technical accuracy with educational accessibility.

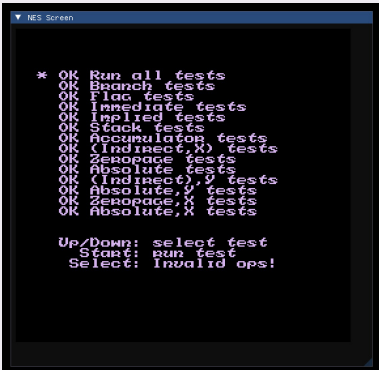
Testing & Validation

The emulator was tested using test and commercial ROMs to ensure correctness and reliability.

CPU instruction test ROMs were used to verify correct implementation of addressing modes and instruction behaviour

PPU tests confirmed accurate rendering of the NES colour palette

The emulator was able to run commercial games, demonstrating functional integration of system components



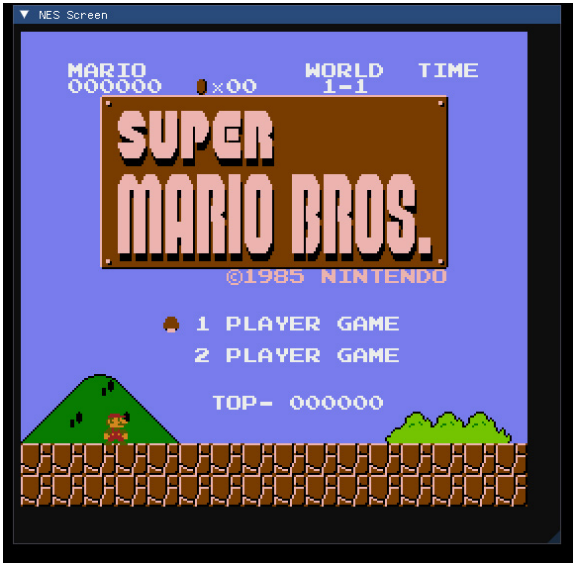
REFERENCES

1. Beck,K., et al. (2001). Manifesto for Agile Software Development
2. Fogleman, B. (2019). Writing a NES emulator in Go.
<https://github.com/fogleman/nest>
3. Gamma, E. (1995). Design patterns : elements of reusable object-oriented software (37th printing.). Addison-Wesley.
4. NESDev Wiki (N.D) –
https://www.nesdev.org/wiki/Nesdev_Wiki

Project Overview

The NES is a widely studied example of early computer architecture, featuring a simple but highly constrained design that makes it suitable for exploring low level system behaviour. Its architecture consists of a CPU, PPU, memory, and cartridge based storage, all interconnected via a shared bus (NESDev Wiki, n.d.). This structure provides a clear model for understanding fundamental computing concepts such as memory mapping, instruction execution, and hardware abstraction.

Existing emulators often prioritise performance and compatibility, sometimes at the expense of transparency (Fogleman, 2019). As a result, they can be difficult to use as educational tools. This project instead focuses on visibility, enabling users to observe how instructions are executed and how graphical data is processed in real time.

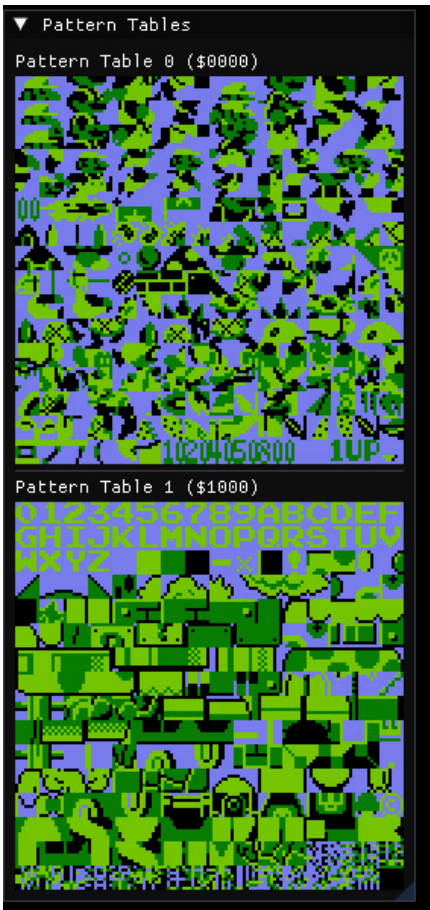


System Design & Implementation

The emulator is based on a modular architecture that mirrors the structure of the original NES hardware. A central bus system connects the CPU, PPU, APU, RAM, and cartridge, allowing each component to operate independently while communicating through a shared interface. This approach reflects real hardware design and makes the system easier to debug and extend.

Memory access is coordinated through the bus, which maps CPU addresses to the correct hardware component (e.g., RAM, PPU registers, or cartridge memory). This abstraction simplifies implementation while preserving the behaviour of the original system, particularly important for handling features such as memory mirroring and mapper controlled ROM access.

The Picture Processing Unit (PPU) is responsible for rendering graphics. It converts tile and attribute data into pixel values using pattern tables and palette memory. Backgrounds and sprites are processed separately, allowing accurate handling of graphical features such as transparency, priority, and sprite flipping. This separation is essential for replicating how the NES draws layered graphics on screen.



The emulator was implemented in C++, providing efficient low level control over memory and timing. A graphical interface was developed using ImGui, This combination supports both performance and usability, making the system suitable for development and educational purposes.

Evaluation & Future Work

The emulator successfully meets its objective of functioning as both a working system and a teaching tool. The modular design and debugging tools allow users to both play games and learn how the nes operates.

However, some limitations remain. The emulator does not fully replicate cycle accurate timing, and support for advanced cartridge mappers is limited. Additionally, while debugging tools expose internal behaviour, they do assume a small amount prior knowledge from the user.

Future work could focus on improving timing accuracy, expanding mapper support, and enhancing usability. One key improvement would be the addition of an instruction explanation system, where stepping through CPU instructions provides real time explanations of their behaviour. This would significantly improve accessibility and reinforce the emulator’s role as a teaching tool.